

Lab Checkpoint 1: stitching substrings into a byte stream

Due: Sunday, October 5, 11:59 p.m.

Collaboration Policy: Same as checkpoint 0.

0 Overview

For Checkpoint 0, you used an *Internet stream socket* to fetch information from a website and send an email message, using Linux’s built-in implementation of the Transmission Control Protocol (TCP). This TCP implementation managed to produce a pair of *reliable in-order byte streams* (one from you to the server, and one in the opposite direction), even though the underlying network only delivers “best-effort” datagrams. By this we mean: short packets of data that can be lost, reordered, altered, or duplicated. You also implemented the byte-stream abstraction yourself, in memory within one computer. Over the coming weeks, you’ll implement TCP yourself, to provide the byte-stream abstraction between a pair of computers separated by an unreliable datagram network.

★*Why am I doing this?* Providing a service or an abstraction on top of a different less-reliable service accounts for many of the interesting problems in networking. Over the last 40 years, researchers and practitioners have figured out how to convey all kinds of things—messaging and e-mail, hyperlinked documents, search engines, sound and video, virtual worlds, collaborative file sharing, digital currencies—over the Internet. TCP’s own role, providing a pair of reliable byte streams using unreliable datagrams, is one of the classic examples of this. A reasonable view has it that TCP implementations count as the **most widely used** nontrivial computer programs on the planet.

The lab assignments will ask you to build up a TCP implementation in a modular way. Remember the `ByteStream` you just implemented in Checkpoint 0? In the coming labs, you’ll end up convey two of them across the network: an “outbound” `ByteStream`, for data that a local application writes to a socket and that your TCP will send *to* the peer, and an “inbound” `ByteStream` for data coming *from* the peer that will be read by a local application.

This checkpoint contains a “hands-on” component and an implementation component. You might prefer to start the implementation component before the lab session, and do the hands-on component at the lab session. If you are a CGOE student, please use EdStem to coordinate a time with another student to do the hands-on component.

The hands-on component involves multiple moving parts—so there might be some glitches. Please bear with us at the lab session and we’ll do our best to get it working for everybody. If you see an error message from the <https://cs144.net> website, please report it in a public post on EdStem and we’ll take a look.

实验检查点 1：将子字符串拼接成字节流

截止时间：10 月 5 日（星期日）晚上 11:59

合作政策：与检查点 0 相同。

0 概述

在检查点 0 中，你使用互联网流套接字从网站获取信息并发送电子邮件，这依赖于 Linux 内置的传输控制协议（TCP）实现。尽管底层网络仅提供“尽力而为”的数据报服务（即可能丢失、乱序、篡改或重复的短数据包），但该 TCP 实现成功创建了一对可靠的按序字节流（一个从你到服务器，另一个反向传输）。同时，你也在单台计算机的内存中自行实现了字节流抽象。在接下来的几周里，你将亲手实现 TCP 协议，在由不可靠数据报网络连接的两台计算机之间提供字节流抽象。

·我为什么要做这个？在另一个不太可靠的网络服务之上提供一种服务或抽象层，正是网络领域诸多有趣问题的来源。过去 40 年间，研究人员和实践者已经摸索出如何通过互联网传递各种内容—消息与电子邮件、超链接文档、搜索引擎、音视频、虚拟世界、协作文件共享、数字货币。TCP 本身的作用—利用不可靠的数据报提供一对可靠的字节流—就是这类问题的经典案例。一个合理的观点认为，TCP 实现堪称地球上使用最广泛的非平凡计算机程序。

实验作业将要求你以模块化方式构建一个 TCP 实现。还记得你在检查点 0 中刚刚实现的字节流吗？在后续实验中，你将通过网络传输两个这样的字节流：一个“出站”字节流，用于本地应用程序写入套接字并由你的 TCP 发送给对端的数据；以及一个“入站”字节流，用于接收来自对端并由本地应用程序读取的数据。

此检查点包含一个“动手操作”部分和一个实现部分。你可能更倾向于在实验课之前开始实现部分，并在实验课上完成动手操作部分。如果你是 CGOE 学生，请使用 EdStem 与其他同学协调时间，共同完成动手操作部分。

动手操作部分涉及多个动态环节—因此可能会出现一些小问题。请在实验课上耐心配合，我们会尽力确保每位同学都能顺利完成。如果你在 <https://cs144.net> 网站上看到错误信息，请在 EdStem 上公开发帖报告，我们会及时处理。

1 Getting started

Your implementation of TCP will use the same Minnow library that you used in Checkpoint 0, with additional classes and tests. To get started:

1. Make sure you have committed all your solutions to Checkpoint 0. Please don't modify any files outside of the `src` directory, or `webget.cc`. You may have trouble merging the Checkpoint 1 starter code otherwise.
2. While inside the repository for the lab assignments, run `git fetch` to retrieve the most recent version of the lab assignments.
3. Download the starter code for Checkpoint 1 by running `git merge origin/check1-startercode`.
4. Make sure your build system is properly set up: `cmake -S . -B build`
5. Compile the source code: `cmake --build build`
6. Open and start editing the `writeups/check1.md` file. This is the template for your lab writeup and will be included in your submission.

2 Hands-on component: a private network for the class

We have created a private network for the CS144 class. This will allow your VM to send datagrams directly to and from the VMs of other students in the class. To make your VM join this network:

1. On your VM, install the “wireguard” and “apparmor-utils” packages by running `sudo apt install wireguard apparmor-utils`
2. Run `sudo aa-complain /etc/apparmor.d/wg` to remove some security constraints on the private-networking software.
3. Visit <https://cs144.net/wg> and follow the instructions to join the CS144 private network.
4. Once you have joined the network, verify that you can connect by following the “ping” instructions on that page (the instructions appear after you have joined the network).
5. Every time you reboot your VM, you'll have to rejoin the network (if you want to be able to send datagrams to and from other students in this class). You don't have to register a new public key each time, but you do have to rerun the commands on that webpage. The commands will be the same each time.

1 开始准备

你的 TCP 实现将使用与检查点 0 相同的 Minnow 库，并新增了额外的类和测试。开始操作：

1. 请确保你已将检查点 0 的所有解决方案提交。请不要修改 `src` 目录或 `webget.cc` 之外的任何文件，否则在合并检查点 1 的起始代码时可能会遇到问题。
2. 在实验作业的仓库目录中，运行 `git fetch` 以获取最新版本的实验作业。
3. 通过运行 `git merge origin/check1-startercode` 下载检查点 1 的起始代码。
4. 确保你的构建系统已正确设置：`cmake -S . -B build`
5. 编译源代码：`cmake --build build`
6. 打开并开始编辑 `writeups/check1.md` 文件。这是你的实验报告模板，将包含在你的提交内容中。

2 动手实践环节：为课程搭建的私有网络

我们为 CS144 课程创建了一个私有网络。这将允许你的虚拟机直接与班上其他同学的虚拟机之间发送数据报。要让你的虚拟机加入此网络：

1. 在您的虚拟机上，通过运行 ``sudo apt install wireguard apparmor-utils`` 命令安装“wireguard”和“apparmor-utils”软件包。
2. 运行 ``sudo aa-complain /etc/apparmor.d/wg`` 以解除对私有网络软件的部分安全限制。
3. 访问 <https://cs144.net/wg> 并按照说明加入 CS144 私有网络。
4. 加入网络后，按照该页面上的“ping”说明（该说明在您加入网络后显示）验证您是否能够成功连接。
5. 每次重启虚拟机后，你需要重新加入网络（如果你想与本课程的其他同学互相发送数据报）。无需每次注册新的公钥，但必须重新运行该网页上的命令。每次执行的命令都是相同的。

2.1 Ping a friend and look at the datagrams

- On your own computer (e.g. your Mac or Windows machine—not your VM), install the “wireshark” program by following the instructions at <https://www.wireshark.org/>. (If you are using Debian or Ubuntu GNU/Linux, the command is `sudo apt install wireshark`.)
- Ask a groupmate for their IP address (the one shown on the <https://cs144.net/wg> webpage **for them**). Using the `ping` command, send some “echo request” datagrams to your friend, and make sure that you get some “echo reply” datagrams back.
- Tips:
 - You can end the “ping” program by typing `ctrl-C`.)
 - You can make the “ping” command go faster by including the argument `-i 0.2`. This will make it send an “echo request” every 0.2 seconds (5 times per second).
 - You can make the “ping” command print out a summary of the statistics so far (without ending it) by running this command in another terminal: `killall -QUIT ping`.
- Begin a report in your writeup, including the following information:
 - What is the average round-trip delay between when your VM sends an “echo request” and when it receives an “echo reply” from your groupmate’s VM?
 - What was the delivery rate (what percentage of “echo requests” received a corresponding “echo reply”)? What was the loss rate (this is 100% minus the delivery rate)? Send at least 1,000 pings to get a reliable estimate. (This will take about three minutes if using `ping -i 0.2`.)
 - Did you see any duplicated datagrams (ping will print “DUP”)?
 - While the ping is running, you and your groupmate can capture some of the raw Internet datagrams by running `sudo rm /tmp/capture.raw; sudo tcpdump -n -w /tmp/capture.raw -i wg0 --print --packet-buffered`. This command will capture the datagrams on the “wg0” interface (the private class network) to a file (“/tmp/capture.raw”), while also printing them out to the screen. Make sure you see some “echo request” and “echo reply” lines printed—that indicates your groupmate is receiving your datagrams and replying to you.
 - Use the `wireshark` program to inspect the `/tmp/capture.raw` file on each of your VMs. You probably want to `scp` the `capture.raw` file to your own computer (e.g. a Mac or Windows machine) and then use wireshark to open this file, so you can use its graphical interface. Can you find the fields of the Internet datagram that were discussed in the Jan. 10 (and match the diagram at <https://www.rfc-editor.org/rfc/rfc791.html#page-11>)?
 - Are there any differences between the **same** datagram when it was captured on your VM compared with when they were captured on your friend’s VM? What? Make sure you are really analyzing the **same** datagram, captured from two different points of view (sender vs. receiver).

2.1 向同学发送 Ping 请求并观察数据报

- 在你的个人电脑上 (例如 Mac 或 Windows 设备, 非虚拟机), 按照 <https://www.wireshark.org/> 的说明安装 “Wireshark” 程序。(若使用 Debian 或 Ubuntu GNU/Linux 系统, 命令为 `sudo apt install wireshark`。)
- 向同组同学索要他们的 IP 地址 (即 <https://cs144.net/wg> 网页上显示的地址)。使用 `ping` 命令向同学发送若干 “回显请求” 数据报, 并确保能收到对应的 “回显应答” 数据报。
- 提示:
 - 您可以通过按 `Ctrl + C` 来结束 “ping” 程序。
 - 您可以通过添加参数 `-i 0.2` 来让 “ping” 命令运行得更快。这将使其每 0.2 秒 (即每秒 5 次) 发送一次 “回显请求”。
 - 你可以在另一个终端中运行以下命令, 让 “ping” 命令打印出到目前为止的统计摘要 (而不终止它): `killall -QUIT ping`。
- 在你的报告中开始撰写, 包含以下信息: (a) 你的虚拟机发送 “回显请求” 到从同组同学的虚拟机收到 “回显应答” 之间的平均往返延迟是多少?
 - 投递率是多少 (即 “回显请求” 中收到对应 “回显应答” 的百分比)? 丢包率是多少 (即 100% 减去投递率)? 发送至少 1000 次 ping 以获得可靠估计。(如果使用 `ping -i 0.2`, 这大约需要三分钟。)
 - 你是否看到任何重复的数据报 (ping 会打印 “DUP”) ?
 - 在运行 ping 的同时, 你和你的组员可以通过执行以下命令来捕获部分原始互联网数据报: `sudo rm /tmp/capture.raw; sudo tcpdump -n -w /tmp/capture.raw -i wg0 --print --packet-buffered`。该命令将在 “wg0” 接口 (私有类网络) 上捕获数据报并保存到文件 (“/tmp/capture.raw”), 同时将内容打印到屏幕上。请确保看到若干行 “echo request” 和 “echo reply” 输出—这表明你的组员正在接收你的数据报并向你回复。
 - 使用 wireshark 程序检查每台虚拟机上的 `/tmp/capture.raw` 文件。建议将 `capture.raw` 文件通过 `scp` 传输到你的个人电脑 (例如 Mac 或 Windows 机器), 然后使用 wireshark 打开该文件, 以便利用其图形界面。你能找到 1 月 10 日课程中讨论的互联网数据报字段吗 (这些字段与 <https://www.rfc-editor.org/rfc/rfc791.html#page-11> 的图示相匹配) ?
 - 同一数据报在你的虚拟机上捕获时, 与在你朋友的虚拟机上捕获时是否存在差异? 具体是什么差异?

请确保你确实是在分析同一数据报, 只是从两个不同视角 (发送方与接收方) 捕获的。

2.2 Send an Internet datagram by hand

In the `apps/ip_raw.cc` file, write a program that sends an Internet datagram to your friend by using a raw socket, using the same method as the Sept. 24 lecture. It’s okay to adapt code from this lecture.

1. Send your groupmate an Internet datagram with IP protocol “5” (you’ll have to use “sudo” to run the “./build/apps/ip_raw” program), and have your friend use `tcpdump` to make sure they receive the datagram. They can run `sudo tcpdump -n -i wg0 'proto 5'` to print out only datagrams matching protocol “5”. Make sure they get it!
2. Send your groupmate a user datagram (with IP protocol “17”), using the “user datagram” header format in <https://www.rfc-editor.org/rfc/rfc768>. Have your groupmate **receive** this datagram *without using “sudo”*. They can use the “nc -u” program as was done in lecture, or a C++ program using the `UDPSocket` class—whatever they prefer!
3. Include the code for your “ip_raw.cc” in your submission to this checkpoint.
4. Do the same in reverse and receive a datagram from your groupmate.

3 Implementation: putting substrings in sequence

As part of the lab assignment, you are implementing a TCP receiver: the module that receives datagrams and turns them into a reliable byte stream to be read from the socket by the application—just as your `webget` program read the byte stream from the webserver in Checkpoint 0.

The TCP sender is dividing its byte stream up into short *segments* (substrings no more than about 1,460 bytes apiece) so that they each fit inside a datagram. But the network might reorder these datagrams, or drop them, or deliver them more than once. The receiver must reassemble the segments into the contiguous stream of bytes that they started out as.

In this lab you’ll write the data structure that will be responsible for this reassembly: a **Reassembler**. It will receive substrings, consisting of a string of bytes, and the index of the first byte of that string within the larger stream. **Each byte of the stream** has its own unique index, starting from zero and counting upwards. As soon as the Reassembler knows the **next** byte of the stream, it will write it to the Writer side of a `ByteStream`— the same `ByteStream` you implemented in checkpoint 0. The Reassembler’s “customer” can read from the Reader side of the same `ByteStream`.

Here’s what the interface looks like:

```
// Insert a new substring to be reassembled into a ByteStream.
void insert( uint64_t first_index, std::string data, bool is_last_substring );
```

2.2 手动发送一个互联网数据报

在 `apps/ip_raw.cc` 文件中，编写一个程序，使用原始套接字向你的朋友发送一个互联网数据报，方法与 9 月 24 日的课程相同。可以改编该课程中的代码。

1. 向你的组员发送一个 IP 协议号为“5”的互联网数据报（你需要使用“sudo”权限运行“./build/apps/ip_raw”程序），并让朋友用 `tcpdump` 确认收到该数据报。他们可以运行 `sudo tcpdump -n -i wg0 'proto 5'` 来仅打印匹配协议“5”的数据报。确保他们成功收到!
2. 向你的组员发送一个用户数据报（IP 协议号为“17”），使用 <https://www.rfc-editor.org/rfc/rfc768> 中定义的“用户数据报”头部格式。让组员在不使用“sudo”的情况下接收该数据报。他们可以使用课堂上演示的“nc -u”程序，或基于 `UDPSocket` 类的 C++ 程序—按他们喜好选择即可！
3. 将你的“ip_raw.cc”代码包含在本检查点的提交内容中。
4. 反向操作，从你的组员处接收一个数据报。

3 实现：将子串按顺序拼接

作为实验任务的一部分，你需要实现一个 TCP 接收器：该模块负责接收数据报，并将其转换为可靠的字节流，供应用程序从套接字读取—就像你在检查点 0 中通过 `webget` 程序从 Web 服务器读取字节流一样。

TCP 发送器会将其字节流分割成短段（每个子串不超过约 1460 字节），以便每个段都能放入一个数据报中。但网络可能会对这些数据报进行重新排序、丢弃或重复传输。接收器必须将这些段重新组装成它们最初所构成的连续字节流。

在本实验中，你将编写负责重组数据的数据结构：一个重组器。它将接收子字符串（由字节序列组成）以及该字符串在完整数据流中的起始字节索引。数据流中的每个字节都有其唯一的索引，从零开始递增。一旦重组器确定了数据流的下一个字节，它就会将该字节写入字节流的写入端—即你在检查点 0 中实现的同一个字节流。重组器的“用户”可以从同一字节流的读取端读取数据。

以下是接口定义：

```
// 插入一个新的子字符串，以便重组为字节流。
void insert( uint64_t first_index, std::string data, bool is_last_substring );
```



```
// How many bytes are stored in the Reassembler itself?
// This function is for testing only; don't add extra state to support it.
uint64_t count_bytes_pending() const;

// Access output stream reader
Reader& reader();
```

**Why am I doing this?* TCP robustness against reordering and duplication comes from its ability to stitch arbitrary excerpts of the byte stream back into the original stream. Implementing this in a discrete testable module will make handling incoming segments easier.

The full (public) interface of the reassembler is described by the `Reassembler` class in the `reassembler.hh` header. Your task is to implement this class. You may add any private members and member functions you desire to the `Reassembler` class, but you cannot change its public interface.

3.1 What should the Reassembler store internally?

The `insert` method informs the `Reassembler` about a new excerpt of the `ByteStream`, and where it fits in the overall stream (the index of the beginning of the substring).

In principle, then, the `Reassembler` will have to handle three categories of knowledge:

- 1. Bytes that are the **next bytes** in the stream. The `Reassembler` should push these to the stream (`output.writer()`) as soon as they are known.
- 2. Bytes that fit within the stream’s available capacity but can’t yet be written, because earlier bytes remain unknown. These should be stored internally in the `Reassembler`.
- 3. Bytes that lie beyond the stream’s available capacity. These should be discarded. The `Reassembler`’s will not store any bytes that can’t be pushed to the `ByteStream` either immediately, or as soon as earlier bytes become known.

The goal of this behavior is to **limit the amount of memory** used by the `Reassembler` and `ByteStream`, no matter how the incoming substrings arrive. We’ve illustrated this in the picture below. The “capacity” is an upper bound on *both*:

- 1. The number of bytes buffered in the reassembled `ByteStream` (shown in green), and
- 2. The number of bytes that can be used by “unassembled” substrings (shown in red)

```
// 重组器本身存储了多少字节？
// 此函数仅用于测试；请勿添加额外状态来支持它。
uint64_t count_bytes_pending() const;

// 访问输出流读取器
Reader& reader();
```

·我为什么要这样做？TCP 对乱序和重复数据的鲁棒性，源于其能够将字节流的任意片段重新拼接回原始流。在一个可独立测试的模块中实现这一功能，将使处理传入的数据段变得更加容易。

重组器的完整（公共）接口由 `reassembler.hh` 头文件中的 `Reassembler` 类定义。你的任务是实现这个类。你可以向 `Reassembler` 类添加任何私有成员和成员函数，但不能更改其公共接口。

3.1 `Reassembler` 内部应存储什么内容？

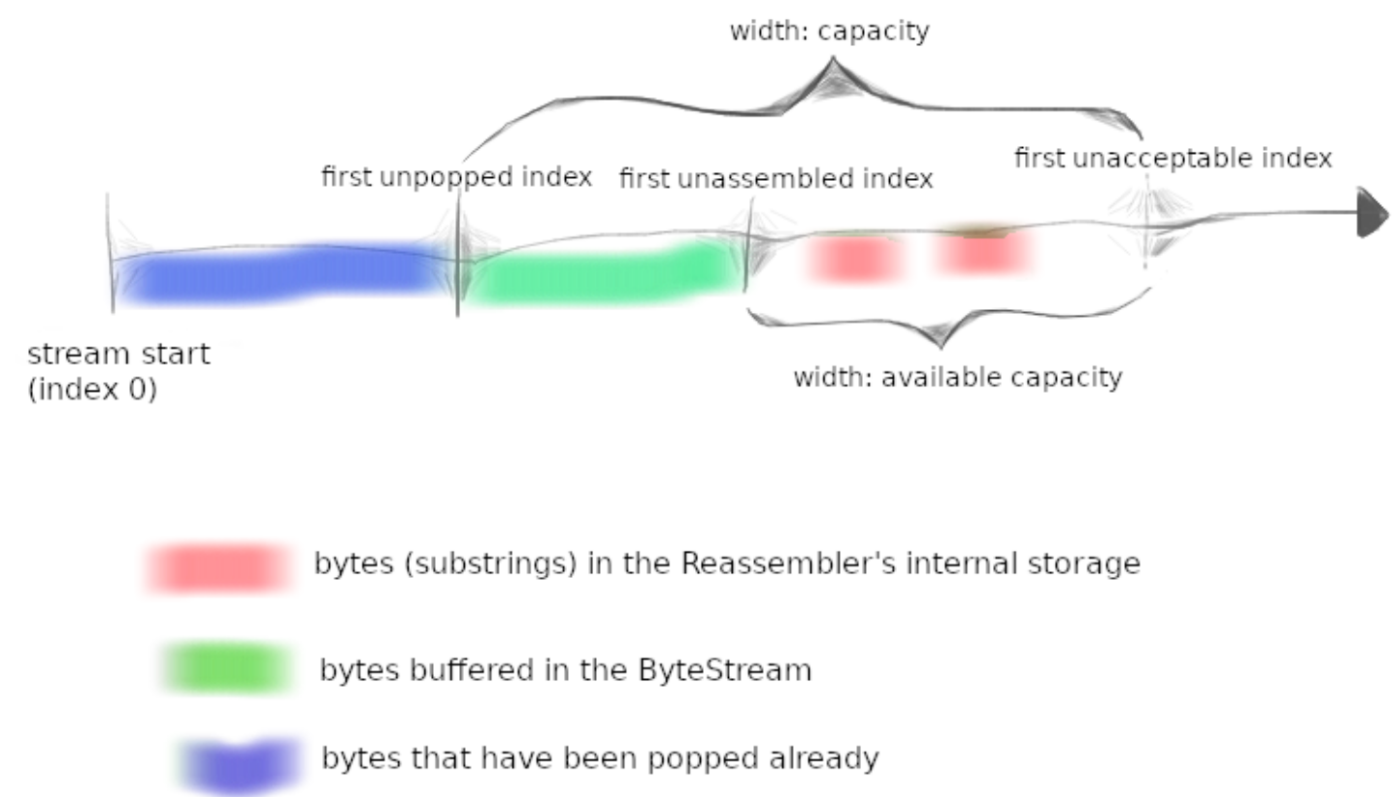
`insert` 方法向 `Reassembler` 告知 `ByteStream` 的一个新片段，以及该片段在整个流中的位置（子串起始处的索引）。

原则上，`Reassembler` 需要处理三类信息：

- 1. 流中接下来的字节。一旦这些字节被确认，`Reassembler` 应立即将其推入流中（通过 `output.writer()`）。
- 2. 位于流可用容量范围内但尚无法写入的字节，因为前面的字节仍未知。这些字节应存储在 `Reassembler` 内部。
- 3. 超出流可用容量的字节应被丢弃。重组器不会存储任何无法立即推送到字节流中的字节，也无法在较早字节变得已知后推送到字节流中。

此行为的目标是限制重组器和字节流使用的内存量，无论传入的子串如何到达。我们在下图中对此进行了说明。“容量”是以下两者的上限：

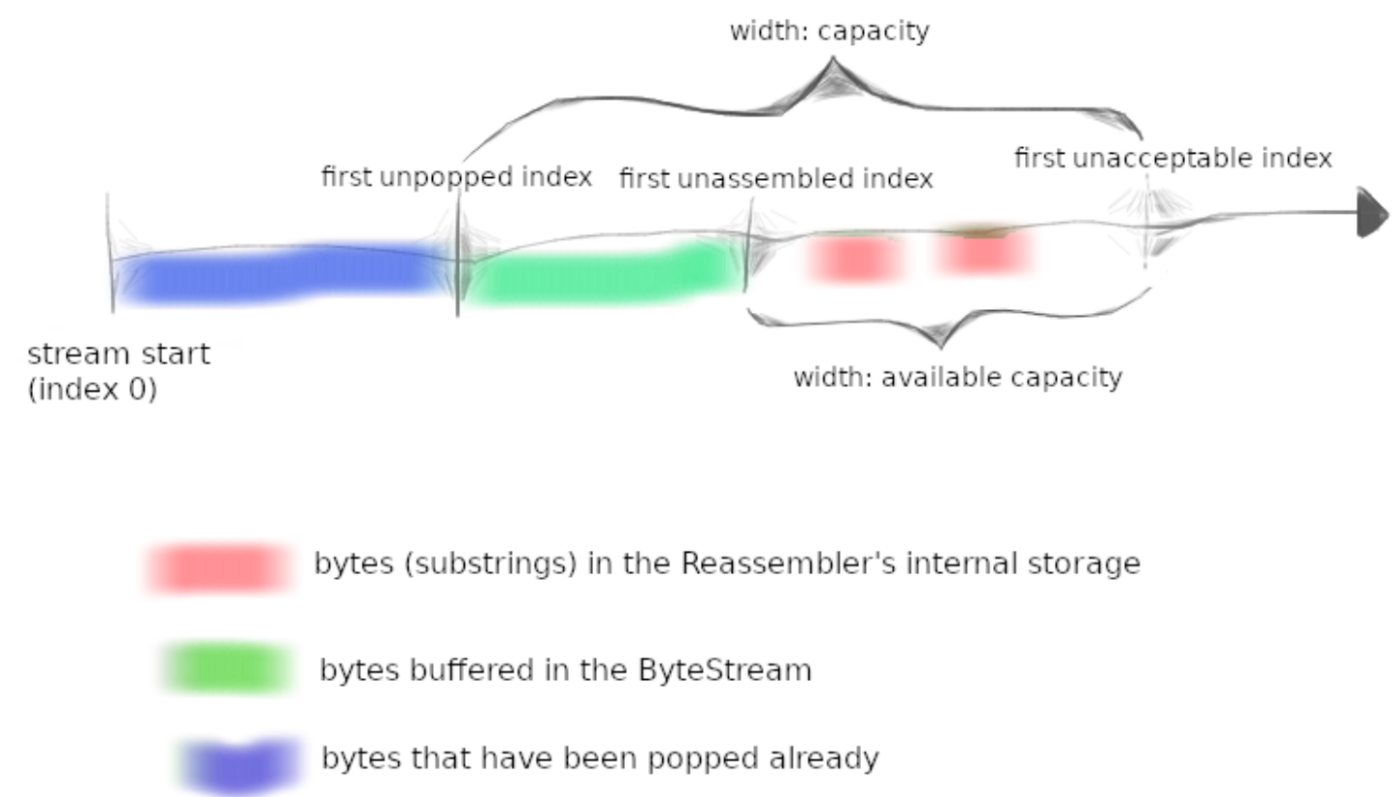
- 1. 已重组字节流中缓冲的字节数（以绿色显示），以及
- 2. 可用于"未组装"子串的字节数（以红色显示）



You may find this picture useful as you implement the `Reassembler` and work through the tests—it's not always natural what the “right” behavior is.

3.2 FAQs

- *What is the index of the first byte in the whole stream?* Zero.
- *How efficient should my implementation be?* The choice of data structure is again important here. Please don't take this as a challenge to build a grossly space- or time-inefficient data structure—the Reassembler will be the foundation of your TCP implementation. You have a lot of options to choose from.
We have provided you with a benchmark; anything greater than 0.1 Gbit/s (100 megabits per second) is acceptable. A top-of-the-line Reassembler will achieve 10 Gbit/s.
- *How should inconsistent substrings be handled?* You may assume that they don't exist. That is, you can assume that there is a unique underlying byte-stream, and all substrings are (accurate) slices of it.
- *What may I use?* You may use any part of the standard library you find helpful. In particular, we expect you to use at least one data structure.



你在实现重组器并通过测试时，可能会发现这张图很有用—因为有时“正确”的行为并不那么直观。

3.2 常见问题解答

- 整个数据流中第一个字节的索引是多少？ 是零。
- 我的实现应该达到怎样的效率？ 数据结构的选择在这里同样重要。请不要把这当作挑战，去构建一个空间或时间效率极低的数据结构—重组器将是你 TCP 实现的基础。你有很多选择。
我们为你提供了一个基准：大于 0.1 Gbit/s (即 100 兆比特每秒) 即为可接受。顶级重组器 (Reassembler) 可实现 10 Gbit/s 的速率。
- 如何处理不一致的子串？ 你可以假设它们不存在。也就是说，你可以假定存在一个唯一的底层字节流，所有子串都是该字节流的（准确）切片。
- 我可以使用哪些工具？ 你可以使用标准库中任何对你有帮助的部分。特别地，我们期望你至少使用一种数据结构。

- *When should bytes be written to the stream?* As soon as possible. The only situation in which a byte should not be in the stream is that when there is a byte before it that has not been “pushed” yet.
- *May substrings provided to the `insert()` function overlap?* Yes.
- *Will I need to add private members to the Reassembler?* Yes. Substrings may arrive in any order, so your data structure will have to “remember” substrings until they’re ready to be put into the stream—that is, until all indices before them have been written.
- *Is it okay for our re-assembly data structure to store overlapping substrings?* No. It is possible to implement an “interface-correct” reassembler that stores overlapping substrings. But allowing the re-assembler to do this undermines the notion of “capacity” as a memory limit. If the caller provides redundant knowledge about the same index, the Reassembler should only store one copy of this information.
- *Will the Reassembler ever use the Reader side of the `ByteStream`?* No—that’s for the external customer. The Reassembler uses the Writer side only.
- *How many lines of code are you expecting?* When we run `./scripts/lines-of-code` on the starter code, it prints:

```
ByteStream:    90 lines of code
Reassembler:   26 lines of code
```

and when we run it on our solutions, it prints:

```
ByteStream:   109 lines of code
Reassembler:   87 lines of code
```

- So a reasonable implementation of the Reassembler might be about 50–60 lines of code for the Reassembler (on top of the starter code).
- *More FAQs:* For more, please see https://cs144.github.io/lab_faq.html.

4 Development and debugging advice

1. You can test your code (after compiling it) with `cmake --build build --target check1`.
2. Please re-read the section on “using Git” in the Lab 0 document, and remember to keep the code in the Git repository it was distributed in on the `main` branch. Make small commits, using good commit messages that identify what changed and why.
3. Please work to make your code readable to the CA who will be grading it for style and soundness. Use reasonable and clear naming conventions for variables. Use comments to explain complex or subtle pieces of code. Use “defensive programming”—explicitly

- 字节应何时写入流中？应尽快写入。唯一不应将字节写入流的情况是当该字节之前存在尚未被“推送”的字节时。
- 提供给 `insert()` 函数的子字符串可以重叠吗？可以。
- 我是否需要向 Reassembler 添加私有成员？需要。子字符串可能以任意顺序，因此你的数据结构必须“记住”子串，直到它们准备好被放入流中——也就是说，直到它们之前的所有索引都已被写入。
- 我们的重组数据结构可以存储重叠的子串吗？不可以。虽然可以实现一个“接口正确”的重组器来存储重叠的子串，但允许重组器这样做会破坏“容量”作为内存限制的概念。如果调用者提供了关于同一索引的冗余信息，重组器应仅存储该信息的一份副本。
- 重组器会使用字节流的读取端吗？不会——那是给外部客户使用的。重组器仅使用写入端。
- 你预计需要多少行代码？当我们运行 `./scripts/lines-of-code` 时在起始代码中，它输出：

```
ByteStream: 90 行代码
Reassembler: 26 行代码
```

而当我们运行我们的解决方案时，它输出：

```
ByteStream: 109 行代码
Reassembler: 87 行代码
```

- 因此，一个合理的重组器实现（在起始代码基础上）大约需要 50 到 60 行代码。
- 更多常见问题：请参阅 https://cs144.github.io/lab_faq.html。

4 开发与调试建议

1. 你可以在编译代码后，使用 `cmake --build build --target check1` 来测试你的代码。
2. 请重新阅读实验 0 文档中关于“使用 Git”的部分，并记得将代码保留在最初分发时所在的 Git 仓库的主分支上。进行小规模提交，使用能清晰说明更改内容和原因的优质提交信息。
3. 请努力使你的代码对负责评分风格和合理性的助教具有可读性。为变量使用合理且清晰的命名规范。使用注释解释复杂或微妙的代码片段。采用“防御性编程”——明确地

- check preconditions of functions or invariants, and throw an exception if anything is ever wrong. Use modularity in your design—identify common abstractions and behaviors and factor them out when possible. Blocks of repeated code and enormous functions will make it hard to follow your code.
- Please also keep to the “Modern C++” style described in the Checkpoint 0 document. The cppreference website (<https://en.cppreference.com>) is a great resource, although you won’t need any sophisticated features of C++ to do these labs. (You may sometimes need to use the `move()` function to pass an object that can’t be copied.)
 - If you get your builds stuck and aren’t sure how to fix them, you can erase your `build` directory (`rm -rf build`—please be careful not to make a typo as this will erase whatever you tell it), and then run `cmake -S . -B build` again.

5 Submit

- In your submission, please only make changes to the `.hh` and `.cc` files in the `src` directory. Within these files, please feel free to add private members as necessary, but please don’t change the *public* interface of any of the classes.
- Before handing in any assignment, please run these in order:
 - Make sure you have committed all of your changes to the Git repository. You can run `git status` to make sure there are no outstanding changes. Remember: make small commits as you code.
 - `cmake --build build --target format` (to normalize the coding style)
 - `cmake --build build --target check1` (to make sure the automated tests pass)
 - Optional: `cmake --build build --target tidy` (suggests improvements to follow good C++ programming practices)
- Write a report in `writeups/check1.md`. This file should be a roughly 20-to-50-line document with no more than 80 characters per line to make it easier to read. The report should contain the following sections:
 - Structure and Design.** Describe the high-level structure and design choices embodied in your code. You don’t need to discuss in detail what you inherited from the starter code. Use this as an opportunity to highlight important design aspects and provide greater detail on those areas for your grading TA to understand. What data structures did you choose in your header file? Are any of them not *strictly* necessary? We’d like you to avoid redundant state if at all possible, unless you think and can justify that there’s a serious performance penalty from doing so. You are strongly encouraged to make this writeup as readable as possible by using subheadings and outlines. Please do not simply translate your program into an paragraph of English.

- 检查函数的前置条件或不变量，如果出现任何错误则抛出异常。在设计中使用模块化—识别通用的抽象和行为，并尽可能将其提取出来。重复的代码块和庞大的函数会使你的代码难以理解。
- 请同时遵循 Checkpoint 0 文档中描述的“现代 C++”风格。cppreference 网站 (<https://en.cppreference.com>) 是一个很好的资源，尽管你不需要使用 C++ 的任何复杂特性来完成这些实验。（有时你可能需要使用 `move()` 函数来传递一个无法被复制的对象。）
 - 如果你的构建卡住且不确定如何修复，可以删除构建目录 (`rm -rf build`——请小心不要打错字，否则会删除你指定的任何内容)，然后重新运行 `cmake -S . -B build`。

5 提交

- 在提交时，请仅对 `src` 目录下的 `.hh` 和 `.cc` 文件进行修改。在这些文件中，你可以根据需要自由添加私有成员，但请不要更改任何类的公共接口。
- 在提交任何作业之前，请按顺序执行以下操作：（a）确保已将所有更改提交到 Git 仓库。你可以运行 `git status` 来确认没有未提交的更改。请记住：在编码过程中进行小规模提交。
 - `cmake --build build --target format`（用于规范化代码风格）
 - `cmake --build build --target check1`（用于确保自动化测试通过）
 - 可选: `cmake --build build --target tidy`（提供改进建议，遵循良好的 C++ 编程实践）
- 在 `writeups/check1.md` 中撰写一份报告。该文件应约为 20 至 50 行，每行不超过 80 个字符，以便于阅读。报告应包含以下部分：
 - 结构与设计。** 描述代码中的高层结构及设计选择。无需详细讨论从起始代码继承的内容。借此机会突出重要的设计方面，并为评分助教提供更详细的说明，以便其理解。你在头文件中选择了哪些数据结构？其中是否有并非严格必要的数据结构？我们希望你能尽可能避免冗余状态，除非你认为并能够证明这样做会带来严重的性能损失。

强烈建议你通过使用小标题和大纲来使这份报告尽可能易读。请不要简单地将你的程序翻译成一段英文。

- (b) **Alternative design choices** that you considered or ideally evaluated in terms of their performance, difficulty to write (e.g., hours required to produce a bug-free implementation), difficulty to read (e.g., lines of code and their degree of subtlety or nonobvious correctness), and any other dimensions you think are interesting for the reader (or for your own past self before you did this assignment). Include any measurements if applicable. **We really care about helping you grow your ability to imagine and describe the realistic implementation options and weigh their tradeoffs.**

(c) **Implementation Challenges.** Describe the parts of code that you found most troublesome and explain why. Reflect on how you overcame those challenges and what helped you finally understand the concept that was giving you trouble. How did you attempt to ensure that your code maintained your assumptions, invariants, and preconditions, and in what ways did you find this easy or difficult? How did you debug and test your code?

(d) **Remaining Bugs.** Point out and explain as best you can any bugs (or unhandled edge cases) that remain in the code.
4. In your writeup, please also fill in the number of hours the assignment took you and any other comments.

5. The mechanics of “how to turn it in” will be announced before the deadline.

6. Please let the course staff know ASAP of any problems at the lab session, or by posting a question on Ed. Good luck!

- (b) 你考虑过或理想情况下评估过的替代设计方案，包括其性能、编写难度（例如，生成无错误实现所需的小时数）、阅读难度（例如，代码行数及其微妙程度或非显而易见的正确性），以及你认为对读者（或对完成此作业前的你自己）有意义的其他维度。请一并说明。

如果适用，请提供测量数据。我们非常重视帮助您提升能力，以构想并描述切实可行的实施方案，并权衡其利弊。

(c) 实现挑战。描述你发现最棘手的代码部分并解释原因。反思你是如何克服这些挑战的，以及最终帮助你理解那个困扰你的概念的是是什么。你是如何尝试确保代码维护了你的假设、不变式和前置条件的？在哪些方面你觉得这很容易或困难？你是如何调试和测试代码的？

(d) 剩余缺陷。尽可能指出并解释代码中仍存在的任何缺陷（或未处理的边界情况）。
4. 在你的报告中，请同时填写完成作业所花费的小时数以及任何其他意见。

5. 关于“如何提交”的具体操作方式将在截止日期前公布。

6. 如有任何问题，请尽快在实验课上告知课程工作人员，或通过 Ed 平台发布问题。祝你好运！